

L - 5 Navigation Methods, Locators

Navigation methods

Code Explanation:

```
import {test,chromium} from 'playwright/test';
test('navigation',async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://www.facebook.com/');
  await page.click('//a[text()="Create a Page"]');
  await page.waitForTimeout(2000);
  await page.goBack();
  await page.waitForTimeout(2000);
  await page.goForward();
  await page.waitForTimeout(2000);
  await page.reload();
  await page.waitForTimeout(2000);
})
```

- `<pageName>.goBack()` - Goes to the previous page
- `page.goForward()` - Goes to the next page
- `page.reload()` - Refreshes the page



```
await page.waitForNavigation({
  timeout: 10000,
  waitUntil: 'domcontentloaded'
});
```

Option	Type	Description
timeout	number	Maximum time to wait for navigation.
waitUntil	'load' \	'domcontentloaded' \
		'networkidle'

When to consider navigation finished.

Locator methods (getby..)

```
test("getBy Locators - 1",async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://www.saucedemo.com/v1/');
  await page.getByPlaceholder('Username').fill('Vaidehi');
  await page.waitForTimeout(2000);
})
```

```

    await page.close();
    await browser.close();
  })

```

```

test("getBy Locators - 2", async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://www.w3schools.com/');
  const ltctext = await page.getByRole('heading',{name:'Learn to
Code'}).innerText();
  const linkroleex = await page.getByRole('link',{name: 'Not Sure Where To
Begin?'}).click();
  await page.waitForTimeout(2000);
  console.log(ltctext);
  await page.close();
  await browser.close();
})

```

- **getByRole()**
 - **role:** "alert" | "alertdialog" | "application" | "article" | "banner" | "blockquote" | "button" | "caption" | "cell" | "checkbox" | "code" | "columnheader" | "combobox" | "complementary" | "contentinfo" | "definition" | "deletion" | "dialog" | "directory" | "document" | "emphasis" | "feed" | "figure" | "form" | "generic" | "grid" | "gridcell" | "group" | "heading" | "img" | "insertion" | "link" | "list" | "listbox" | "listitem" | "log" | "main" | "marquee" | "math" | "meter" | "menu" | "menubar" | "menuitem" | "menuitemcheckbox" | "menuitemradio" | "navigation" | "none" | "note" | "option" | "paragraph" | "presentation" | "progressbar" | "radio" | "radiogroup" | "region" | "row" | "rowgroup" | "rowheader" | "scrollbar" | "search" | "searchbox" | "separator" | "slider" | "spinbutton" | "status" | "strong" | "subscript" | "superscript" | "switch" | "tab" | "table" | "tablist" | "tabpanel" | "term" | "textbox" | "time" | "timer" | "toolbar" | "tooltip" | "tree" | "treegrid" | "treeitem"
 - **options?:** { checked?: boolean; disabled?: boolean; exact?: boolean; expanded?: boolean; includeHidden?: boolean; level?: number; name?: string | RegExp; pressed?: boolean; selected?: boolean; }

```

test("Rest of the getBy locator practice", async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://www.facebook.com/');
  await page.waitForTimeout(2000);
  await page.getByTestId('open-registration-form-button').click();
  await page.waitForTimeout(2000);
  console.log(await page.getText('Create a new account').innerText());
  await page.waitForTimeout(2000);
  console.log(await page.getTitle('Sign up for Facebook').innerText());
  await page.getTitle('Click for more information').first().click();
  await page.waitForTimeout(5000);
  await page.getByLabel('Female').first().click();
  await page.waitForTimeout(2000);

```

```
    await browser.close();
  })
```

- `getById` - accesses the element using the attribute data-testid
- `getText` - accesses the element using the visible text
- `getTitle` - accesses the element using the text in the <title> tag &/or title attribute
- `getLabel` - accesses the element using the text in the <label> tag

Absolute path - xpath written from the root element HTML of the document

Syntax -

```
./html[1]/body[1]/div[4]/div[1]/div[1]/div[1]/div[1]/div[1]/div[1]/div[1]/form[1]/table[1]/tbody[1]/tr[2]/td[1]/input[1]
```

Scroll

`window.scrollTo(0,1000)` - X, Y coordinates in pixels for scrolling.

`window.scrollTo(0,document.body.scrollHeight)`

`window.scrollTo(0,document.documentElement.scrollHeight)` -

`document.documentElement.scrollHeight` gives the total length; if given in the function scroll to the end of the page.

`<elementname>.scrollIntoViewIfNeeded(true)` - Scrolls to a specific element as mentioned by the <elementname>.

L - 6 Handling multiple similar elements, Shadow DOM, Descendant elements

Handling multiple similar elements

Code Explanation: Select the required element is located and is counted and traversed through them for print all the elements

```
import {test, chromium} from 'playwright/test';
test("Handling Multiple elements", async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://www.w3schools.com/');
  const h1elements = page.locator('h1');
  const size = await h1elements.count();
  for(let i=0;i<size;i++){
    const element = await h1elements.nth(i).innerText();
    console.log(element);
  }
})
```

- `.count()` - Counts the no. of elements having the same <elementname>

- `.nth(i)` - Selects the i-th element from that list (zero-based index).

Shadow DOM Manipulation

Code Explanation: Locate the first element present on the main page. Store it and use it to locate the next inner element, store it and use it for the next element and so on.

```
import {test, chromium} from 'playwright/test';
test("Shadow DOM", async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://in.puma.com/');
  const body = page.locator('body');
  const menlink = body.locator('//span[text()="Men"]');
  await menlink.hover();
  await page.waitForTimeout(10000);
})
```

Descendant elements

Code Explanation: Descendant element selection process - Parent element <space> Child element & so on

Below code also covers the topic of web tables

```
test("Handling Table & Descendant Combinator (space)", async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
  await page.goto('https://www.iplt20.com/points-table/men');
  const teams = page.locator('tr h2');
  const size = await teams.count();
  for(let i=0;i<size;i++){
    const teamName = await teams.nth(i).innerHTML();
    console.log(teamName);
  }
})
```

L - 7 Attaching screenshot to test report, allure report

Attaching Screenshot to the test report

Code Explanation:

```
import {test, chromium} from 'playwright/test';
test("Attaching SS to Test Report", async()=>{
  const browser = await chromium.launch({headless:false});
  const page = await browser.newPage();
```

```

    await page.goto('https://www.amazon.com/');
    await page.waitForTimeout(2000);
    test.info().attach("Amazon Home Page",{
      body:await page.screenshot(),
      contentType: 'image/png'
    })
  })
})

```

- `test.info()` - Returns information about the currently running test.
- `.attach(<2 parameters>)` - Attach a value or a file from disk to the current test. Some reporters show test attachments. Either path or body must be specified, but not both.
 1. Attachment name
 2. `{}` - Work
 - `body`: - includes the work to be done
 - `contentType`: - mentions the type of the attachment

Allure Report Generation

In the terminal:

`npm install -D allure-playwright`

In the config file:

In the reporter key - `['allure-playwright']`

- After this, run any file so that a folder for allure reports gets created

Allure report generation:

In the terminal:

- `allure generate allure-results --clean -o allure-report` - To generate the report & clear the previous reports
- `allure open allure-report` - To open the allure report in the web server

Allure report generation steps that worked successfully

In the config file: In the reporter key - `['allure-playwright']`

In the terminal:

- `npm install -g allure-commandline --save-dev`
- `allure --version`
- `allure generate allure-results --clean -o allure-report`
- `allure open allure-report`

L - 8 Hooks, Validations & Assertions

Hooks

To avoid writing duplicate code some features are provided in playwright -

beforeEach, afterEach, beforeAll, afterAll

Hook	When It Runs	Common Use Case
beforeAll	Once before all tests in a suite	Launch browser, set up database
afterAll	Once after all tests in a suite	Close browser, clean up global resources
beforeEach	Before every individual test	Navigate to page, log in, reset state
afterEach	After every individual test	Log out, clear cookies, reset mocks

Order of execution - beforeAll -> beforeEach -> afterEach -> afterAll

- `.describe` - For specific number of test cases
- `.serial` - For serial test execution

Code Explanation:

```
import { test, chromium, expect } from '@playwright/test';

test.describe.serial('test suite', () =>{
  test.beforeAll(async () => {
    console.log('database connectivity code');
  });
  test.afterAll(async () => {
    console.log('close connection code');
  });

  test.beforeEach(async () => {
    console.log('navigate to homepage');
  });

  test.afterEach(async () => {
    console.log('close page');
  });

  test('test1', async () => {
    console.log('Test 1');
  });

  test('test2', async () => {
    console.log('Test 2');
  });
})
```

```
test('test3', async () => {
  console.log('Test 3');
});
```

Workers concept

Concept - Worker means a single browser, so if a number is mentioned to it means that on that many browsers the test case will run

Use - For parallel execution

Syntax - `npx playwright test <filename> --workers=<number>`

Code Explanation (Hooks Real Time Usage):

```
import { test, chromium } from '@playwright/test';
let browser;
let context;
let page;
test.describe('test suite', () => {
  test.beforeAll(async () => {
    browser = await chromium.launch();
    context = await browser.newContext();
    page = await context.newPage();
  });
  test.beforeEach(async () => {
    await page.goto('https://www.saucedemo.com/v1');
    await page.fill('#user-name', 'standard_user');
    await page.fill('#password', 'secret_sauce');
    await page.click('#login-button');
  });
  test('test add to cart', async () => {
    console.log('product is added to cart');
  });
  test.afterEach(async () => {
    const menubutton = page.locator("//button[text()='Open Menu']");
    await menubutton.click();
    await page.click("//a[text()='Logout']");
  });
  test.afterAll(async () => {
    await page.close();
    await browser.close();
  });
});
```

Assertions & Validations:

Code Explanation:

```

test('validation learnings', async({page})=>{
  await page.goto('https://www.saucedemo.com/v1/');
  await expect(page).toHaveURL('https://www.saucedemo.com/v1/');
  //await expect(page).toHaveTitle('Swag Labs dssdfsd');

  const loginbutton= page.locator("#login-button");
  await expect(loginbutton).toHaveCSS('background-color', 'rgb(226, 35,
26)');

  const backgroundColor = await loginbutton.evaluate(el =>
    getComputedStyle(el).backgroundColor
  );

  console.log('Background color:', backgroundColor);
})

```

L - 9 Data driven testing

Data driven testing:

Concept - Instead writing the same code lines for multiple data, data driven testing helps in executing the required lines of codes for all test data without the need of repeated code writing.

Code Explanation:

```

import {test, chromium} from '@playwright/test';
const loginData1 = [
  {username:'standard_user', password:'secret_sauce'},
  {username:'locked_out_user', password:'secret_sauce'},
  {username:'problem_user', password:'secret_sauce'},
  {username:'performance_glitch_user', password:'secret_sauce'}
]
for (const data of loginData1){
  test(`login test for ${data.username}`, async()=>{
    const browser = await chromium.launch({headless:false});
    const page = await browser.newPage();
    await page.goto('https://www.saucedemo.com/v1/');
    await page.fill('#user-name', data.username);
    await page.fill('#password', data.password);
    await page.waitForTimeout(2000);
    await page.click('#login-button');
  })
}

```

- The above code will run 4 times as there are 4 test data.

Reading & sending data from Excel file:

Installation of XLSX library: npm install xlsx

Code Explanation:

```
import * as XLSX from 'xlsx';
import {test, chromium} from '@playwright/test';
type PersonData = {
  Name: string,
  Age: number,
  Email: string
}
//Load the workbook
const workbook = XLSX.readFile('C:/Users/2430147/OneDrive -
Cognizant/Documents/Test/tests/test.xlsx');
//Select the 1st sheet
//const sheetName = workbook.SheetNames[0];
//console.log(sheetName);
const sheet = workbook.Sheets['Sheet1'];
//Convert sheet to JSON
const testData = XLSX.utils.sheet_to_json<PersonData>(sheet);
//Use the data in PW test
for(const row of testData){
  test(`Test for ${row['Name']}`, async ({page})=>{
    console.log(row['Name'], row['Age'], row['Email']);
    //Add assertions here
  })
}

// Send the data into XLSX through Playwright tests
for (const row of testData) {
  test(`Send Test data of ${row['Name']}`, async () => {
    console.log(row['Name'], row['Age'], row['Email'])
    // Add assertions here
    const browser=await chromium.launch({headless:false});
    const page=await browser.newPage();
    await page.goto('https://www.saucedemo.com/v1/');
    await page.fill('#user-name', row['Name']);
    await page.fill('#password', `${row['Email']}`);
    await page.waitForTimeout(3000);
    await page.click('#login-button');
  });
}
```